Visual Computing

Coordinate Systems

Points aka Vertices

- Start by making a point …

Points

Coordinate Systems

Vectors

Transforms

Projection

Points aka Vertices

- And here it is…



Points

Coordinate Systems

Vectors

Transforms

Projection

Coordinate Systems



P(x,y,z)

Points

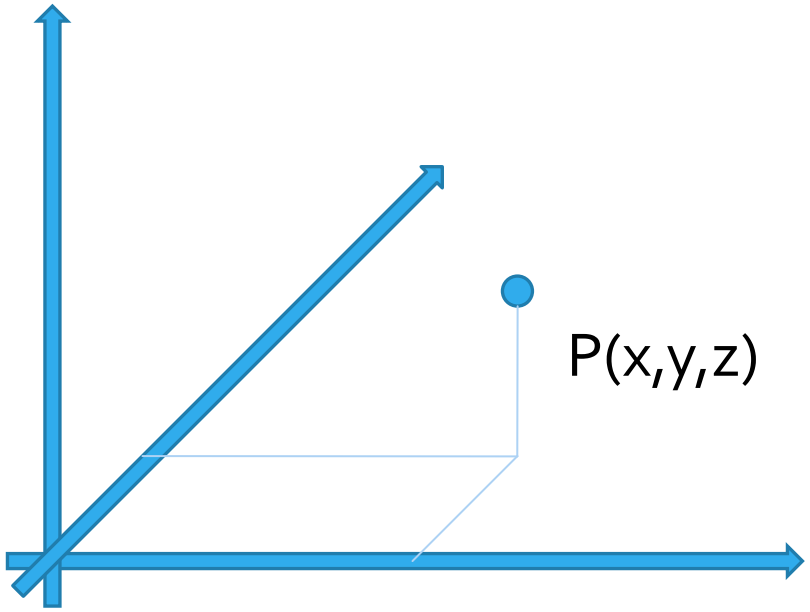
Coordinate Systems

Vectors

Transforms

Projection

Coordinate Systems



Points

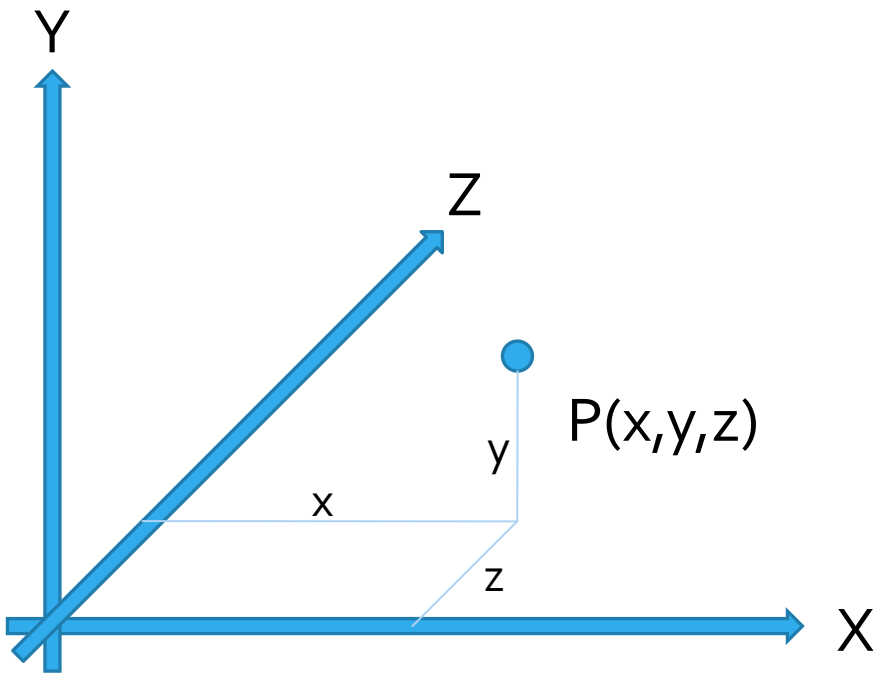
Coordinate Systems

Vectors

Transforms

Projection

Coordinate Systems



Points

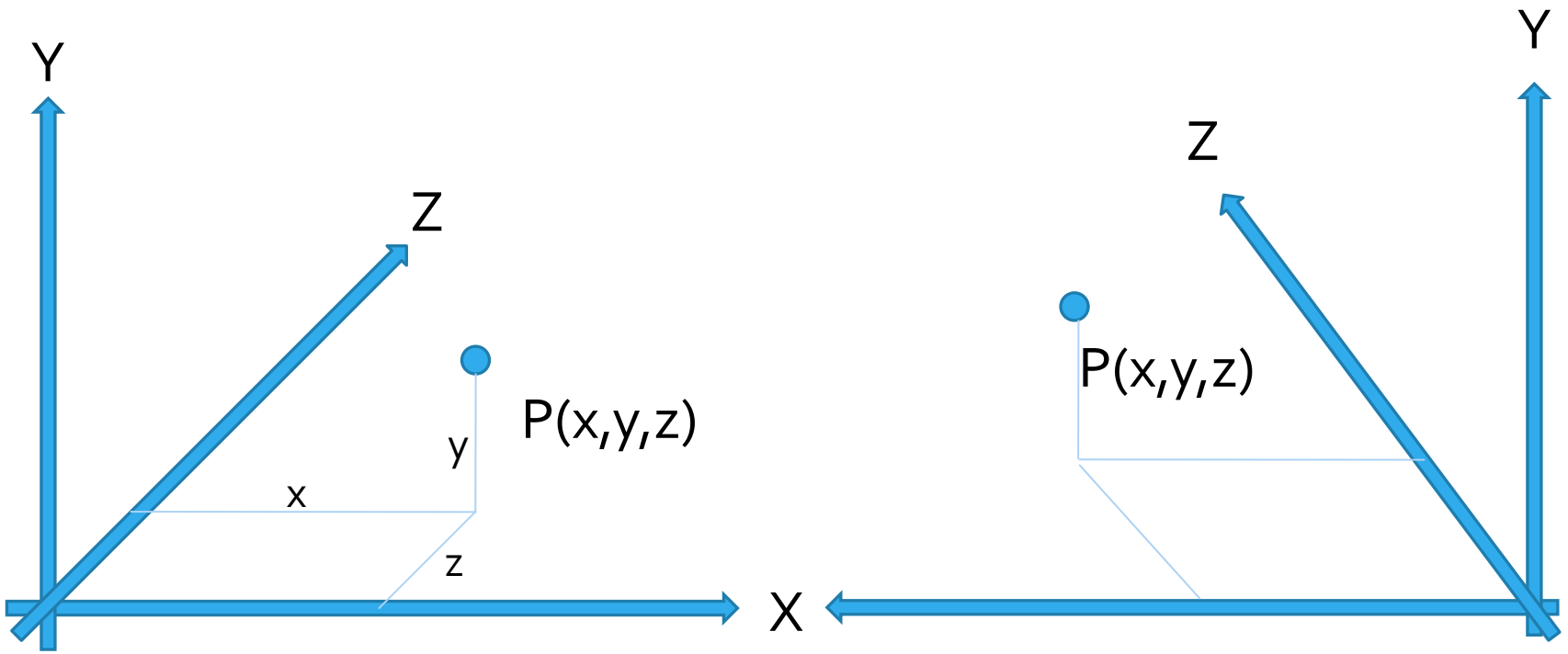
Coordinate Systems

Vectors

Transforms

Projection

Left-handed vs Right-handed



Points

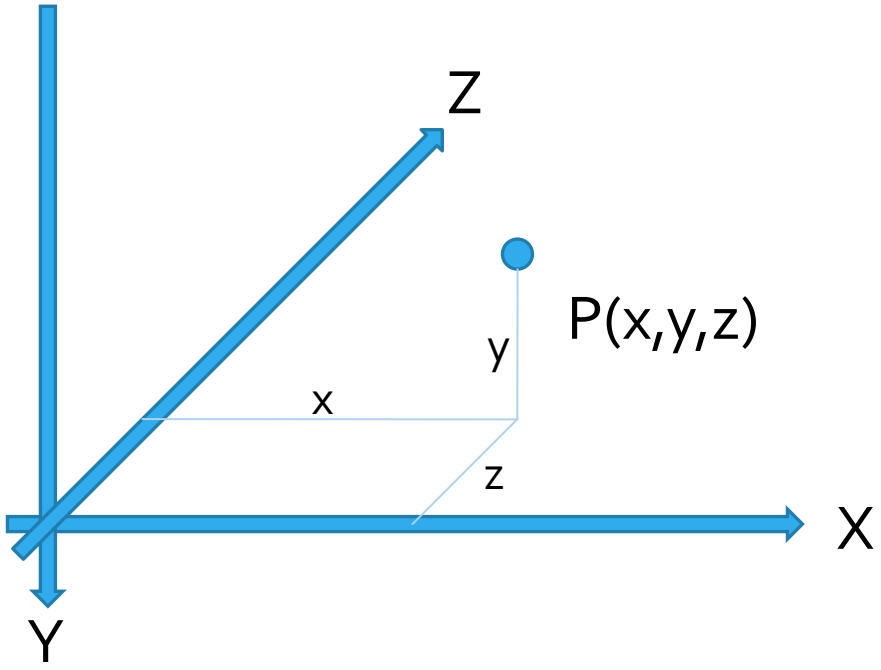
Coordinate Systems

Vectors

Transforms

Projection

Left-handed vs Right-handed



Points

Coordinate Systems

Vectors

Transforms

Projection

The Point



The point is this – remember which way is up.
It will save you hours of debug time.

Points

Coordinate Systems

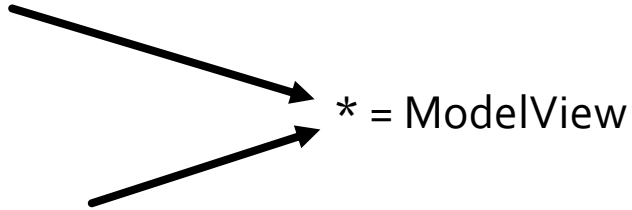
Vectors

Transforms

Projection

Coordinate Spaces

- Object Space
- World Space
- Camera / View Space
- Screen Space



Points

Coordinate Systems

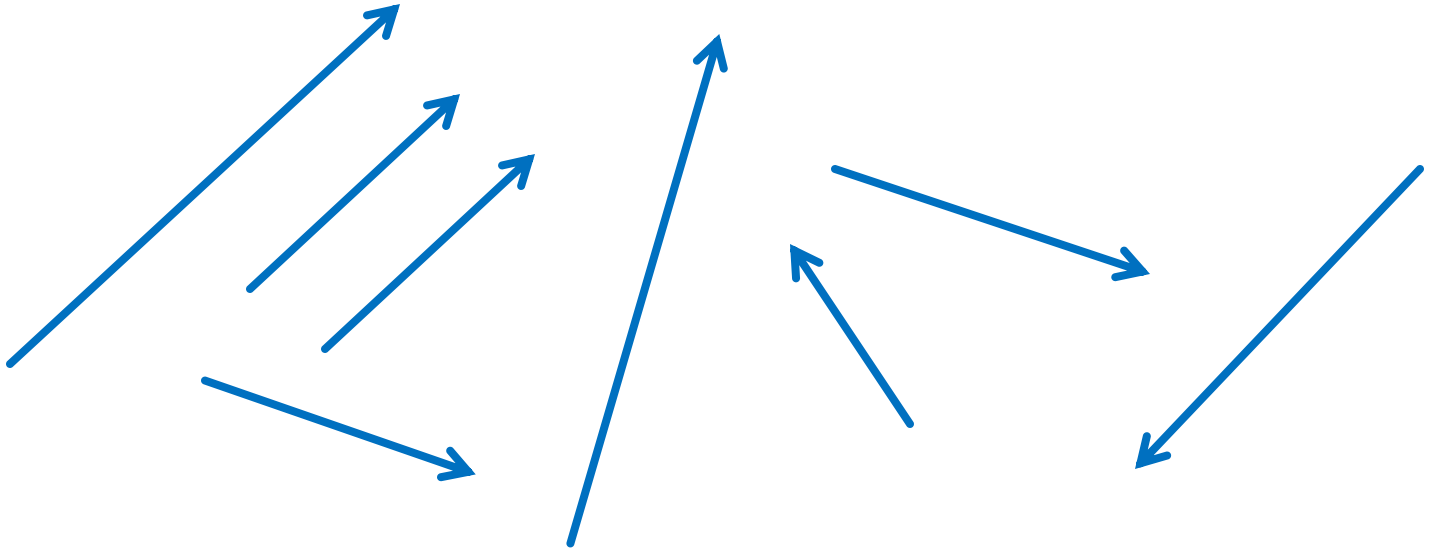
Vectors

Transforms

Projection

Vectors

- Not the same thing as a vertex/point.
- Consists of a direction and a length.



Points

Coordinate Systems

Vectors

Transforms

Projection

Vector Operations

- Add $a+b$ $(a_x+b_x, a_y+b_y, a_z+b_z)$
- Subtract $a-b$ $(a_x-b_x, a_y-b_y, a_z-b_z)$
- Multiply ab $(a_x*b_x, a_y*b_y, a_z*b_z)$
- Scale sa $(s*a_x, s*a_y, s*a_z)$
- Magnitude $|a|$ $\text{sqrt}(a_x^2+a_y^2+a_z^2)$

- Dot Product $a.b$
- Cross Product axb

Points

Coordinate Systems

Vectors

Transforms

Projection

Dot and Cross Products

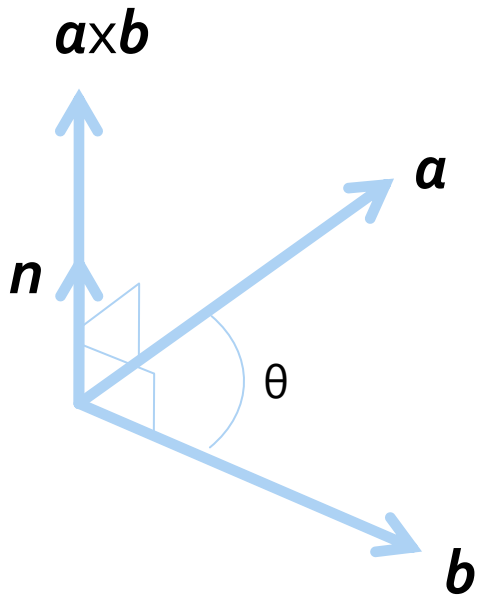
- Dot Product

a.b = |a||b|cos(θ)

$a.b = \sum a_n * b_n$

If a and b are unit length,

Then a.b = cos(θ)



Very useful for lighting calculations.

Points

Coordinate Systems

Vectors

Transforms

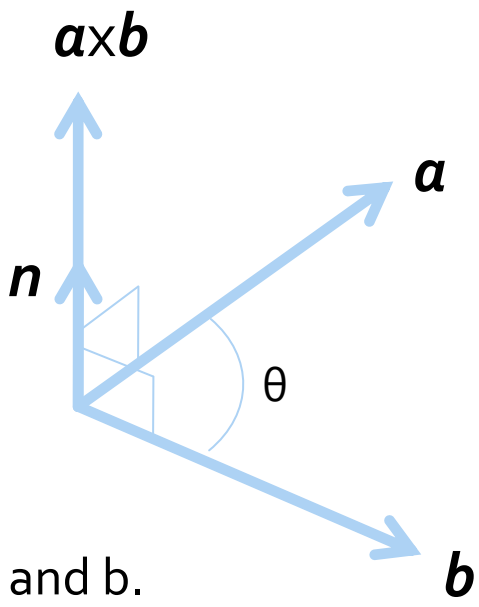
Projection

Dot and Cross Products

- Cross Product

$a \times b = ab \sin(\theta)\, n$

$a \times b = (a_y b_z - a_z b_y,\ a_z b_x - a_x b_z,\ a_x b_y - a_y b_x)$



Creates a vector a right angles to a and b.

Useful for constructing a basis.

Points

Coordinate Systems

Vectors

Transforms

Projection

Transforms

- We can apply a transform to a vertex or a vector.
- We can think of this as transforming something in a coordinate space.
- Or, as a move between coordinate spaces.
  - For example, moving something from object space to world space to view space to screen space. i.e. rendering.
  - Or, from image space to camera space to world space. I.e. scene reconstruction.

Points

Coordinate Systems

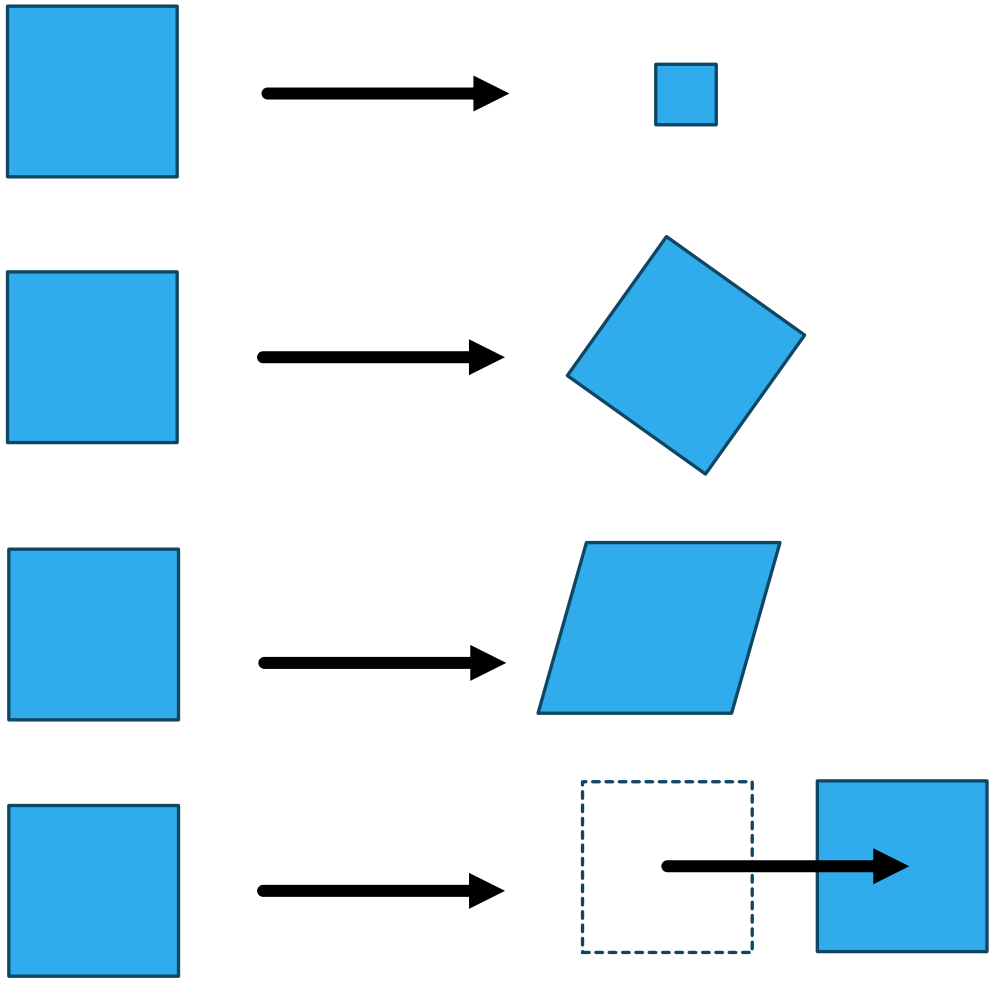
Vectors

Transforms

Projection

Transforms

- Scale (also reflection)
- Rotate
- Shear/Skew
- Translate (Vertices)



Points

Coordinate Systems

Vectors

Transforms

Projection

Matrices (Starting in 2D space)

$P_n = MP_o$

$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ Scale $= \begin{bmatrix} s & 0 \\ 0 & s \end{bmatrix}$ Shear $= \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$

$R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$ $T =$???

Points

Coordinate Systems

Vectors

Transforms

Projection

3D Homogenous

$P_n = MP_o$ where $P_o = (x_o, y_o, 1)$, adding a new coordinate w.

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad \text{Scale} = \begin{bmatrix} s & 0 & 0 \\ 0 & s & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad T = \begin{bmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{bmatrix}$$

Points

Coordinate Systems

Vectors

Transforms

Projection

Concatenate Transforms

$P_n = M_2 M_1 M_0 P_0 = M_n P_0$ where $M_n = M_2 M_1 M_0$

$$M_n = \begin{bmatrix} r_{11} & r_{12} & t_x \\ r_{21} & r_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Points

Coordinate Systems

Vectors

Transforms

Projection

3D Transforms

- Same approach used in 3D using 4D homogenous coordinates. $P = [x,y,z,w]^T$

$$M_n = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Points

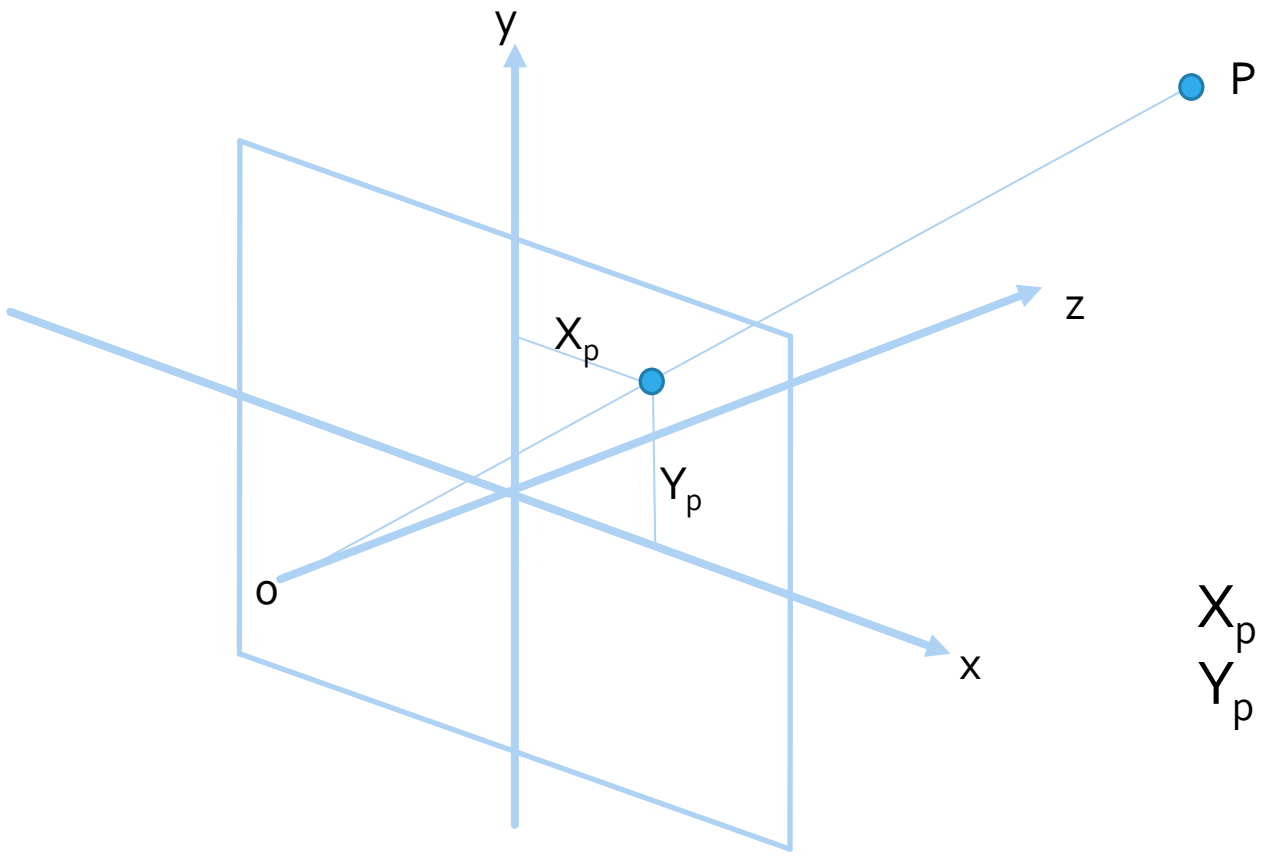
Coordinate Systems

Vectors

Transforms

Projection

Simple Perspective



$X_p = P_x/P_z$
$Y_p = P_y/P_z$

Points

Coordinate Systems

Vectors

Transforms

Projection

Projection Matrix

$$M_p = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$[X\,Y\,Z\,W]^T = M_p\,P$$

$$X = X,\ Y = Y,\ Z = Z,\ W = Z$$

$$(X/W, Y/W, Z/W, W/W) = (X_p, Y_p, 1, 1)$$

Points

Coordinate Systems

Vectors

Transforms

Projection

Projection Matrix

- We want to control the width (angle).
  - Add a scaling factor to the matrix.
- And in graphics application add near and far clipping.
  - Only points in viewing frustum should be visible.

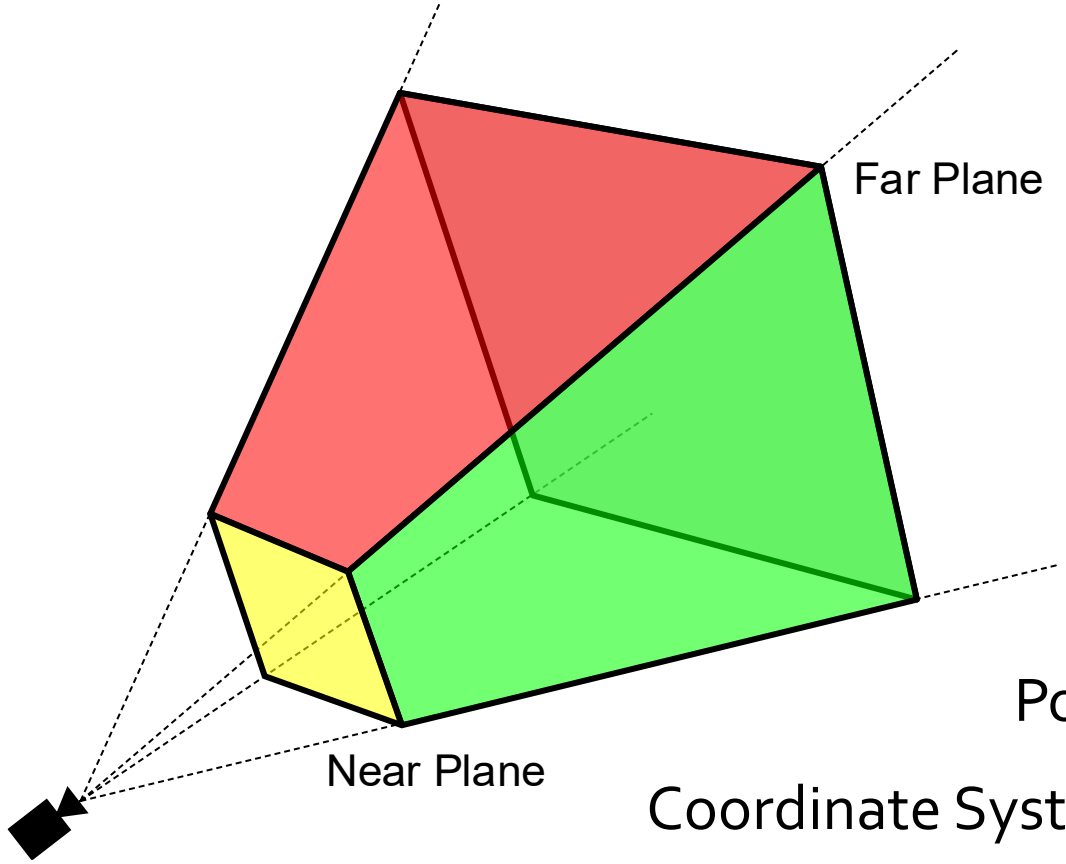


Image: WebGL viewing frustrum, Mozilla CC_BY_SA

Points

Coordinate Systems

Vectors

Transforms

Projection

More details:
https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API/WebGL_model_view_projection

WebGL Perspective

```
side = 500;
var pMatrix = glMatrix.mat4.create();
glMatrix.mat4.perspective(pMatrix, 45, 1, 0.1, 10000.0);
```

pMatrix =

1.794	0	0	0
0	1.794	0	0
0	0	-1	-1
0	0	-0.2	0

Orthographic Projection

- Sometimes, such as when we are drawing 2D we don't want perspective projection.

- We want orthographic projection. I.e. projection on to the z plane.

$$M_p = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

WebGL Ortho

```
side = 500;
var pMatrix = glMatrix.mat4.create();
glMatrix.mat4.orthoNO(pMatrix,-side/2.0,side/2.0,-side/2.0,side/2.0,1.0,-1.0,1.0);
```

$$pMatrix = \begin{bmatrix} 0.004 & 0 & 0 & 0 \\ 0 & 0.004 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Transformation Stack

- We want to concatenate transforms to build more complex transforms.
- It's useful to be able to control the scope of transforms when building a model hierarchy.
- A stack with push and pop operations provides this scoping mechanism.

Points

Coordinate Systems

Vectors

Transforms

Projection

Advanced Computer Graphics

Transforms